

Year 9 CAT 2 2026

Overview

In this assignment you will use and expand upon your knowledge of statistics and polynomials such as linear equations and quadratics. You will also be introduced to the notion of mathematical optimisation.

Permitted tools

1. Python 3 and its standard library, including time and csv.
2. Visual Studio
3. Matplotlib for all graphs and visual displays.
4. No additional external library is required. Ask your teacher before adding one.
5. Copilot.

Responsible AI Use

Although permitted to use generative AI in the form of Copilot, you are expected to do so in such a manner in which your learning and comprehension is complemented, as opposed to outsourced to the AI model. Using generative AI to produce sections of code or suggest methods is appropriate, whereas using it to produce large sections you do not comprehend is not. You should ensure that you are understanding the mathematical concepts as you progress as you will be given a brief quiz on these as part of your grading.

Use of Pre-Existing Libraries

For the statistical measures task, the statistical result must come from logic you write. Loops, arithmetic, comparisons, indexing, slicing, list methods such as append/insert, and len() are allowed. Using functions from pre-existing libraries is not allowed.

Submission

As per usual, you will need to submit your code and a PowerPoint. This brief will guide you as to what to include in the PowerPoint. The due date is Friday Week 2 of Term 4 4:30 pm. You will submit via compass.

Statistical Measures (3 marks)

Implement functions that calculate the following value for a list of numerical data:

- mean
- median
- minimum
- maximum
- range
- lower quartile
- upper quartile
- interquartile range
- variance
- standard deviation

Requirements

Coding:

- Python file named statistics_library.py containing the functions.

PowerPoint:

- None

Data Set Transformations (8 marks)

Now that you have built a small library that will generate statistical measures of data. You will investigate how such measures are impacted by systematic changes to the data. Below is a list of all the text files you will need to analyse with your statistics library. For this part of the assignment build a table that lists the statistical measures along the header row, and then data set names along the first column, and then in the corresponding cell, describe how the measure was changed for this dataset. It should be noted that all datasets are a modification of normal.txt. For example, maybe the median of plus_constant.txt was 100 less than the median of normal.txt, so you could write “Change -100”.

Datasets:

- normal.txt
- plus_constant.txt
 - Plus 100 to each value in normal.txt.
- times_constant.txt
 - Multiply each value of normal.txt by 2.
- extreme_outlier.txt
 - Replacing the max value of normal.txt by 10000.

- duplicated_values.txt
 - Duplicating each value of normal.txt.
- reflected_about_4000.txt
 - Subtracting each value in normal.txt from 4000.
- ax_b.txt
 - Applying the rule $3x+50$ to each value in normal.txt.

Requirements

Coding:

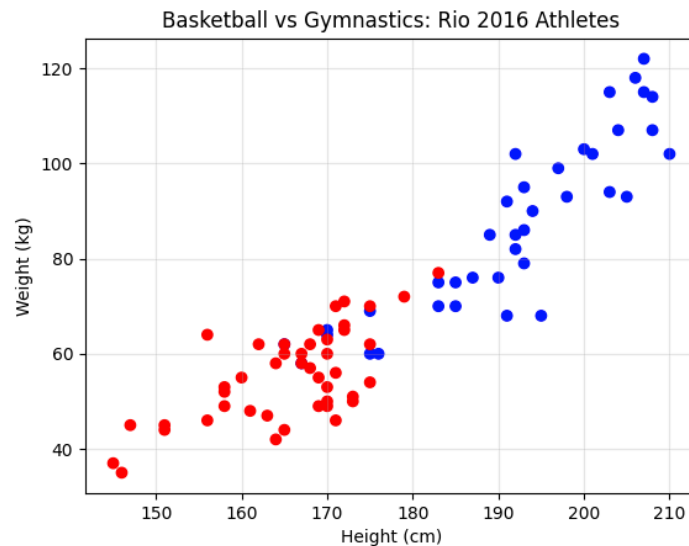
- None.

PowerPoint:

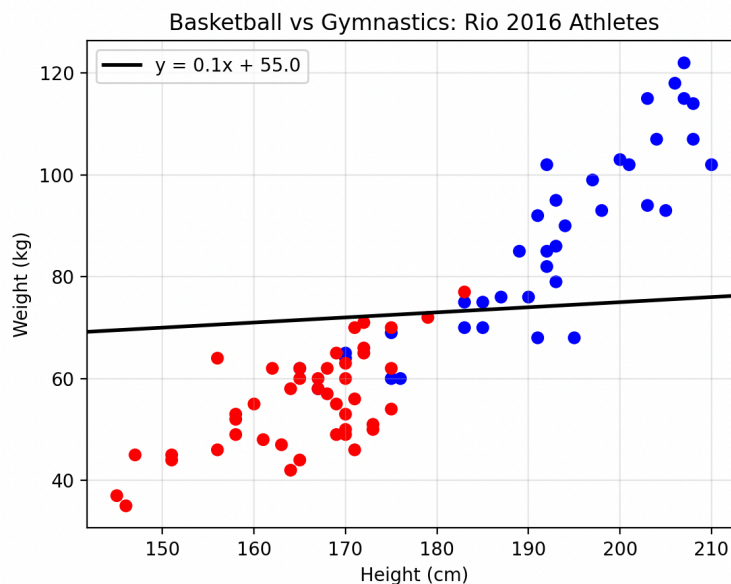
- A table showing how each statistics measure changed depending upon which text file values it was generated from.

Linear Classification of Data

The following shows height and weight values of a set of basketball players(blue) and gymnasts(red) from the Rio 2016 Olympics. As you can see there is a rough trend in the data, the gymnasts are light and shorter, and the basketball players are taller and heavier, however there is some overlap.



Is there a rule we could provide that tells us, based on this data, whether a person should be classified as a basketball player or gymnast? A line may be able to do this for us.



As we can see a rough rule could be used to determine if someone is more likely to be a gymnast or basketball player based on their weight and height.

If $0.1h + 55 \geq w$, then they are probably a gymnast.

If $0.1h + 55 < w$, they are probably a basketball player

We can see this rule is not perfect as some of the basketball players(blue) are below the line and our linear classification rule would judge them as being gymnasts. However clearly we can see the rule is quite effective.

Linear Classification Displaying Data (2 marks)

Write a program that will graph linear_1.csv on a scatterplot using matplotlib. You should automatically determine the range for each axis by using your max and min functions from earlier. Each line of linear_1.csv has three related values, the x value, y value and an associated colour of either red or blue. Groupings of data such as this are commonly referred to as triples. Your graph should display each triple with its x and y-coordinates and the point should be colored accordingly.

Requirements

Coding:

- A python file named data_display.py

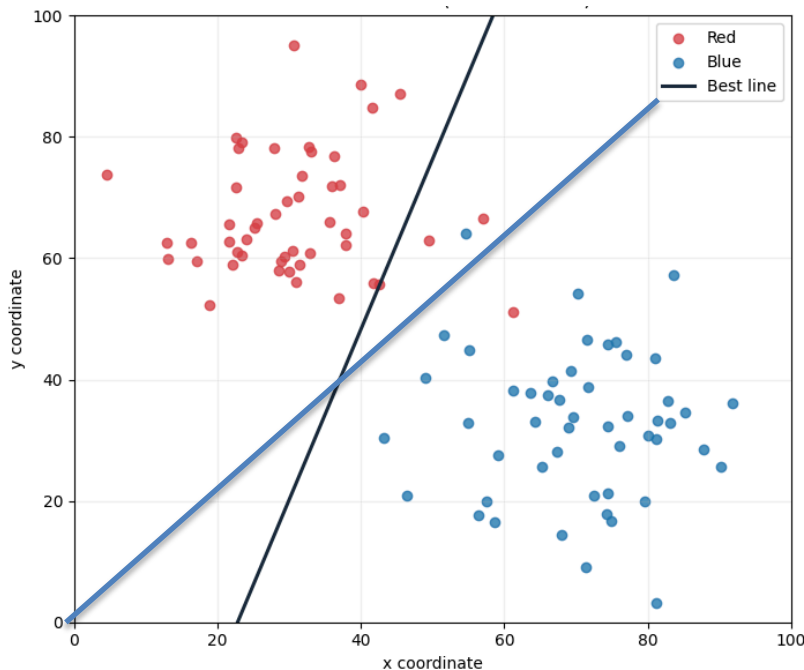
PowerPoint:

- Screenshots of data_display.py displaying data.

Linear Classification - Manual (8 marks)

Consider the following dot plot with a graphed line that is separating the data. This is one of many such lines we could draw to separate the data. For example, if we made the gradient of the line slightly less and adjusted the y intercept, we would get the blue line, which also separates the data. So, we can say there are many ways to separate this data with a line.

The problem is, there will be infinite many lines to separate this data and which we will use to classify points as red or blue. We want an objective way to judge lines so maybe we can find the “best” line.



Write a program that will graph linear_1.csv on a scatterplot. Your graph should colour the dots according to the third value of each triple. Your program should display the data, then allow the user to input an m and c value to provide the gradient and y intercept of the line. Your program should then redisplay the data with your line and should output the percentage of datapoints which are classified correctly by your line (this is how we judge how good our line is).

The rule for determining is the following:

- Setup the line.
- Choose one half of the line as red and the other as blue.
- Count the number of points on the correct side. I.e. how many red points are on the red side and how many are on the blue side. Add these counts and divide by the total number of points. This is the score for this line with the colours on these respective sides. (Score 1)

- Switch the colours to the other side and count the number of points on the correct side. Add these counts and divide by the total number of points. This is the score for this line with the colours on these respective sides. (Score 2)
- Whichever is higher out of Score 1 and Score 2 becomes the score for this line.

Requirements

Coding:

- A python file named manual_classify.py.

PowerPoint:

- Screenshots of manual_classify.py displaying data and a line.

Linear Classification - Random Search (5 marks)

In the previous task, linear classification occurred by user decision. In this task you will automate the process.

Update your code from manual_classify.py so that the value for your line, m and c are randomly selected. Your code should randomly choose values for m and c and then judge how effective these values are by comparing how effectively the data points have been separated. Your code should allow a user to input a number of trials. For example, 10 trials would result in 10 separate random selections for m and c which are all compared, with only the best values for m and c being kept (use the method you used above for judging how effective a line is).

In your PowerPoint include screenshots and a description of how you changed the previous code to randomly search for m and c.

Requirements

Coding:

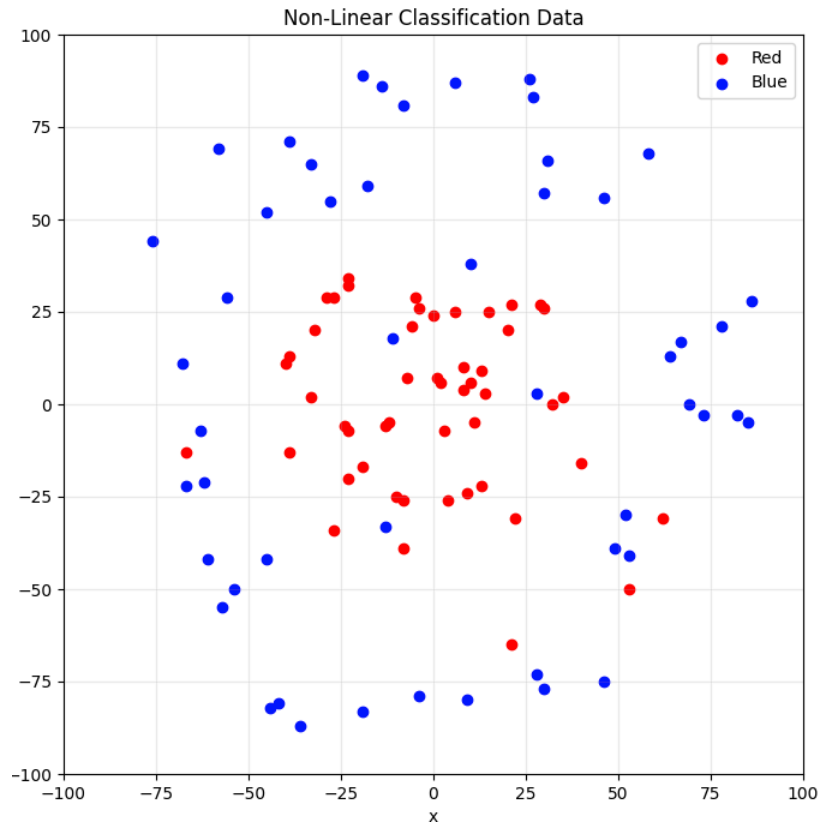
- A python file named auto_classify.py.

PowerPoint:

- Screenshots of auto_classify.py displaying data and a line.
- An explanation of the changes you made.

Non Linear Classification - Manual (8 marks)

Consider the following dot plot with scattered points. Clearly, we cannot use a line to broadly classify the data. But we could possibly use a non-linear relationship such as a circle.



Write a program that will load in the data stored in “non_linear_1.csv”. Your program should display the data and then allow the user to enter the radius of a circle which will classify the data. It should then output a percentage score that tells how effectively the points were classified (Use the same scoring method as you used for linear classification).

Note: For this activity, you may assume that the all x values of any data points you deal with will be between -100 and 100 inclusive, and all y values of any data points you deal with will be between -100 and 100 inclusive.

Requirements

Coding:

- A python file named manual_non_linear.py.

PowerPoint:

- Screenshots of manual_non_linear.py displaying data and a circle.

Non Linear Classification - Automatic (5 marks)

Adjust your program but now make the calculation of the radius automatic. Determine an effective algorithm for this and be willing to justify it. Aim for a superior search algorithm than used for the task “random search linear classification” which used only random trial and testing.

Requirements

Coding:

- A python file named auto_non_linear.py.

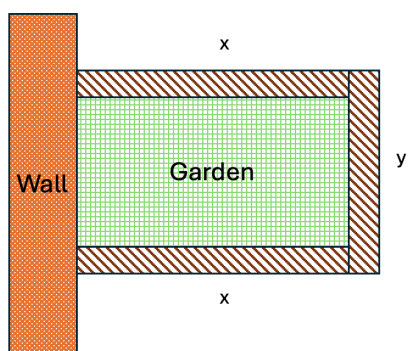
PowerPoint:

- Screenshots of auto_non_linear.py displaying data and a line.
- An explanation of the changes you made.

Optimisation – Manual Graphing (12 marks)

Optimisation is when values for variables are searched for so that a certain criterion may be met. For example, you may want to find values for one or more variables so that the value of another variable is either maximised or minimised. In this assignment, we will only focus on maximisation problems.

The following scenario illustrates the process of maximisation. A rectangular garden is built against a straight wall, so fencing is only required for the other three sides. There are **40 metres** of fencing available. The gardener wants the garden to be as big as possible (he wants to maximise it given the constraints).



Let x be the length of each side perpendicular to the wall. Let y be the length of the side parallel to the wall.

Can we represent this situation with some equations? Do we have any constraints? Yes, the sum of the fence lengths must be 40, therefore we will be working with the following equation:

$$(1) 40 = 2x + y$$

What else? We care about the area (A) of this garden, which is length times width, so we will be working with the following equation too:

$$(2) A = xy$$

This should look familiar to you, these are simultaneous equations. However, there is a difference to what you have encountered in class so far. Normally for two distinct equations you have only two variables, x and y , so you can solve for x and y . But we have 2 equations and 3 variables, so we cannot solve for x and y using our standard approaches.

However, we can still solve for the x and y values we want – reminder they are the values that we want to select so as to maximise A . This gives the gardener the largest garden possible.

We rearrange equation 1 so that either x or y is the subject, in this case I will pick y .

Working follows:

$$(1) 40 - 2x = 2x + y - 2x$$

$$(1) 40 - 2x = y$$

Now we substitute $40 - 2x$ into A in place of y .

$$(2) A = xy$$

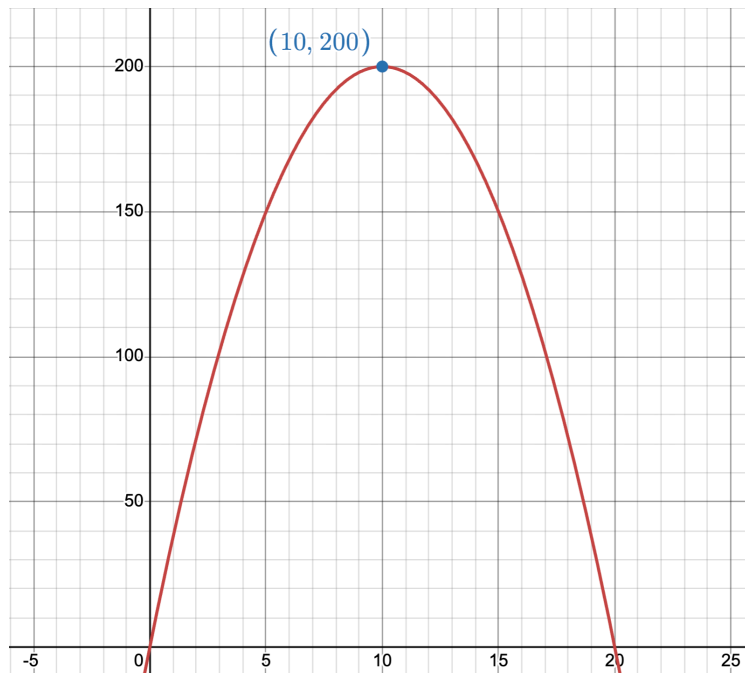
$$(2) A = x(40 - 2x)$$

What type of equation is A ? It is a quadratic...this may be clearer if we expand it:

$$(2) A = x(40 - 2x) = 40x - 2x^2$$

What are the properties of quadratics? They have 0, 1 or 2 x intercepts, they have a y -intercept, they are symmetrical, and they have either a maximum or minimum turning point. The turning point is what we care about.

In this case, as the coefficient of x^2 in this quadratic is negative the turning point will be a maximum. If we can find the x value for that turning point, that will give us the garden width we want. We can then determine the garden length that goes with this width. The method we will use for finding this x value is to simply graph the quadratic as has been done below.



So, reading the quadratic, we can answer the question: what dimension gives the largest possible garden? This graph has Area on the y axis and side length x on the x axis. We see an x value of 10 gives us the maximum Area of 200. To then get the y value which gives us this maximum area we go back to our initial equation (1) and sub in $x=10$ and solve for y :

$$x=10$$

$$(1) 40 = 2x + y = 2(10) + y = 20 + y$$

$$y=20$$

Final answer: Dimensions of $x=10$, $y=20$ produce the maximum area of 200.

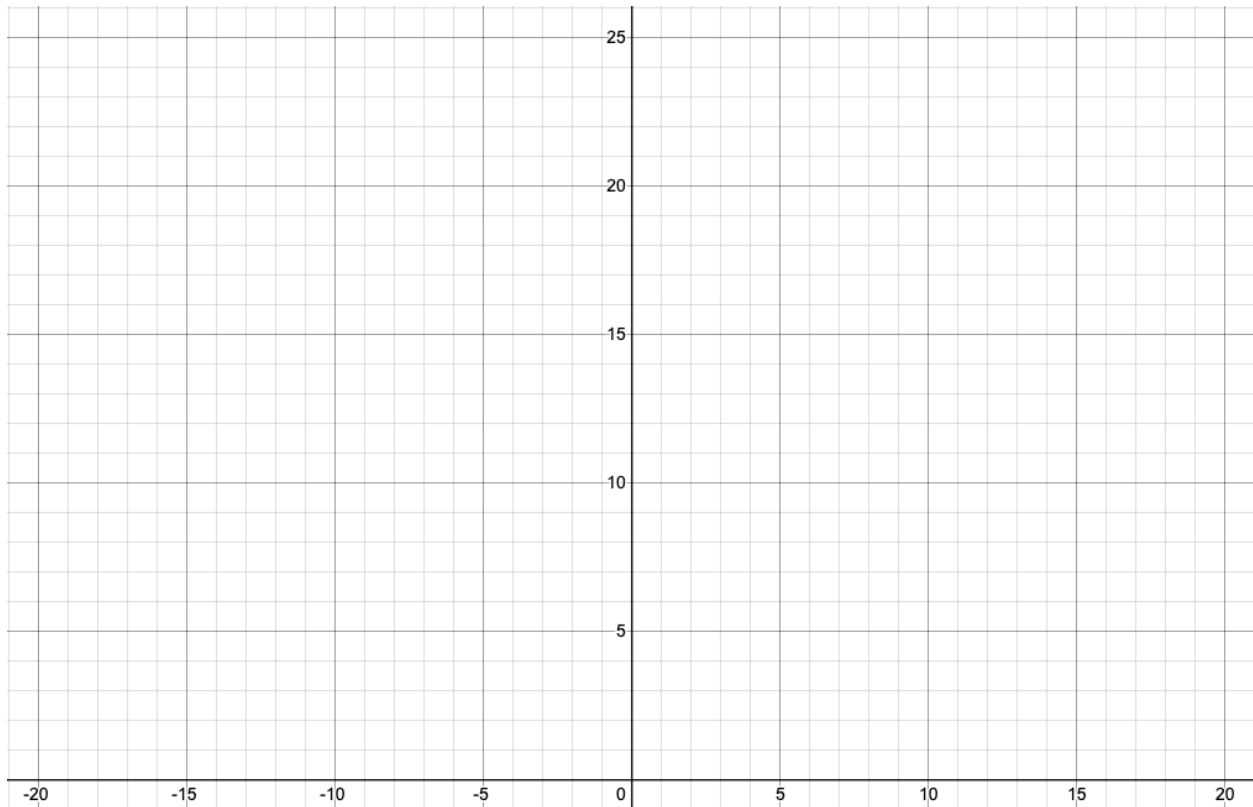
Use the above worked example to solve the following problem using the same methods

A school has 20 metres of fencing to build a rectangular enclosure for its chickens. All four sides of the enclosure must be fenced. You must determine the dimensions of the enclosure that will maximise the area. Let the length of the enclosure be x and y be the width. You must include the graph of the quadratic you come up with.

Step 1 – Generate two equations using the variables of x , y and A :

Step 2 – Rearrange one equation so you can write your A formula as a function of only one variable (referred to as your subject variable later).

Step 3 – Graph your A formula (Hand draw it, but feel free to use Desmos to determine the graph you should draw):



Step 4 – Use the Graph to determine the value of your subject variable which maximises the Area.

Step 5 – List the values for x and y which maximise A. List A also.

Requirements

Coding:

- None.

PowerPoint:

- Include scan(s)/photo(s) of the above working and solutions. Note, your working must be hand written.

Optimisation - Graphing Quadratics (4 marks)

Write a program that will take in a quadratic equation and graph it on matplotlib. For example, the user should be able to enter the above equation in the following form: $y=40x-2x^2$ and have this be graphed and displayed by matplotlib. Your program should allow the user to update the scale of the graph through input, i.e. you should be able to set the min and max values for the x and y axes. You may find using the `eval()` function useful for this task. For example, see the following code below which you are welcome to copy and edit (note this code is incomplete and will need additions):

```
import matplotlib.pyplot as plt

equation = "x**2 - 4"

x_values = []
y_values = []

x = -10

while x <= 10:
    x_values.append(x)
    y_values.append(eval(equation))
    x += 0.1

plt.plot(x_values, y_values)
plt.xlabel("x")
plt.ylabel("y")
plt.grid()

plt.autoscale()

plt.show()
```

Requirements

Coding:

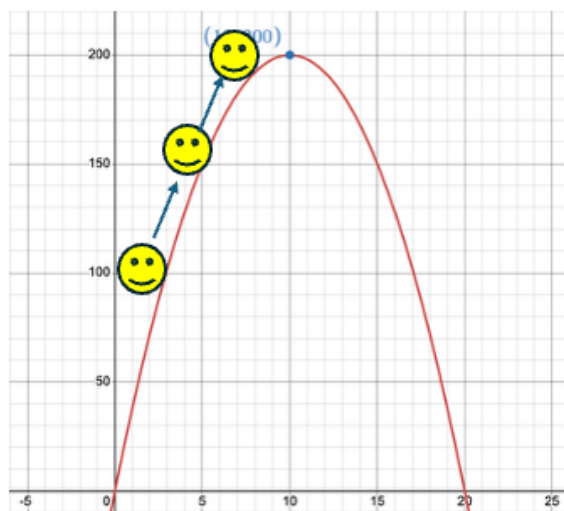
- A python program named `graphing.py` which graphs an arbitrary quadratic.

PowerPoint:

- None

Optimisation - Hill Climbing (10 marks)

Previously you were optimizing values by solving for the x and y values that would maximise some third variable (A) and you used a manual graphing approach. I.e. you graphed a quadratic and looked at the maximum point and used the coordinates of that point to determine the x and y values that generate the maximum of A . Now you will build a program that automatically “climbs” to the maximum of the quadratic, and saves you from having to do any manual graphing. This approach is known as “hill-climbing” and it is a highly respected technique for solving optimisation problems in the wild.



You must build a program which does the following:

- Take in any quadratic equation with a maximum turning point. So, the x^2 term must have a negative coefficient. Then use hill climbing to find the x value of the turning point (Hint: modify your code for “Graphing Quadratics” for this section). The x value you find will only be an approximation, which is okay. It does not have to be the exact x value of the turning point. However it must be a reasonable approximation which we will define as an approximation that is within a distance of $\frac{1}{\sqrt{|a|}}$ where a is the coefficient of the quadratic term in the quadratic equation - $y = ax^2 + bx + c$.
- You do not need to overthink this rule, it just gives us a way to encourage you to think about how you can make your hill climber more accurate. For example, the climber may pass the peak, which means you could have your algorithm turn around and change the step size it has been taking.
- If you are not sure where to begin, watch this video:
 - <https://www.youtube.com/watch?v=kOFBnKDGtJM>

- Your program must allow the user to input any quadratic with a maximum turning point. Then it must select a random starting location (x value) on the quadratic from which it will climb towards the maximum. The path it takes should somehow be shown on a final graph of the equation which you show. Note, you can assume that all quadratics you will be provided with will have a turning point within the following interval: $-1000 \leq x \leq 1000$.
- Your program should output the x value it has found which corresponds to the approximated maximum y value of the quadratic. It should output the y value that goes with this x value also.
- Note: This task is very specifically about finding a maximum approximation using the hill climbing algorithm. Using algebraic techniques such as $-b/2a$ to find the maximum are excluded, as these techniques do not work for higher dimension problems whereas hill climbing does.

Requirements

Coding:

- A Python program named climber.py that meets the above requirements.

PowerPoint:

- A description of your algorithm with explanation of how you find the peak and ensured reasonable accuracy.

Validation Quiz (15 marks)

You will sit a brief 5 question quiz covering the concepts covered in this assignment. The only expected preparation is to have worked through the assignment and considered the concepts and completed the activities. This quiz will not require you to read or write code.

Git (20 marks)

As with past assignments you are expected to use Git to track your progress. At the outset of your project, you should initialise a git repository in a folder named in the following form:

FirstName_Surname_9_Cat_2

So John Smith's folder will be named as follows: John_Smith_9_Cat_2. You will initialise a git repository in this folder and it will be where you store all your assignment files. When working through this assignment, you may decide whether you produce one big program that has different modules that complete separate sections, or whether you make a smaller python program for each section.

OFFICIAL

You should regularly make commits as you progress through the assignment. Each commit should have a description of what progress you have made. Frequent and detailed commits are required to earn marks for this section. Consideration will also be given to use of comments and code readability.